A Relational Graph Policy for Gap-Free Admission in Autonomous Intersection Management

Author Names Affiliation to be added

October 8, 2026

Abstract

Signal-free autonomous intersection management (AIM) can be posed as online admission control over a set of conflict zones: a vehicle is admitted only if its entire route can be reserved back-to-back, so that it never stops inside the intersection. We study this *gap-free admission* problem and present a learned baseline, a relational graph convolutional network (R-GCN) policy trained with proximal policy optimization (PPO). The controller state is a heterogeneous graph of pending vehicles, committed vehicles, and conflict zones joined by seven typed relations that encode route membership, same-lane precedence, shared-zone conflict, and committed blockers. The policy either admits one currently feasible vehicle or defers, while a common feasibility layer enforces zone exclusivity, same-lane first-in-first-out order, and a commitment window. In paired synthetic experiments over eight aggregate Poisson arrival rates (0.10–1.20 vehicles/s; 45 episodes per rate and policy), the R-GCN reduced the rate-averaged mean waiting time by 10.6% relative to the iGreedy earliest-start rule, by 14.4% relative to a Backpressure policy, and by 27.5% relative to LIFO. It was the best policy from 0.65 vehicles/s upward, with a 23.8% reduction relative to iGreedy at 1.20 vehicles/s, but it did not outperform iGreedy under light traffic, where it defers too often. A same-information receding-horizon CP-SAT reference is reported for context. We position the R-GCN as a compact, trainable baseline for learned AIM controllers rather than a dominant policy.

Keywords: autonomous intersection management, connected and automated vehicles, conflict-zone scheduling, graph neural networks, reinforcement learning, relational graph convolution

1 Introduction

Signal-free autonomous intersection management (AIM) has been proposed as an alternative to fixed-cycle signals for connected and autonomous vehicles, with the potential to improve throughput. Realizing that potential depends substantially on the online controller that sequences individual vehicles. Recent work has therefore explored learned controllers, often trained with reinforcement learning (RL) and compared with a greedy earliest-achievable-start rule or a queue-based heuristic. Because such comparators can be weak, an apparent gain may overstate the value of the additional model complexity.

This paper studies a stricter version of the control problem and a learned baseline for it. In *gap-free admission*, a vehicle is admitted only if every conflict zone on its route can be reserved back-to-back, so the vehicle passes through the intersection at its free-flow pace and never waits inside it. Each admitted vehicle is therefore described by a single departure time, and the entire delay is paid before entry. The controller must decide, as vehicles become visible, which of the currently admissible vehicles to admit next and when to hold back; admissions that are not yet locked can be revised when new vehicles are detected. The combinatorial structure is that of a multi-resource sequencing problem, which relational graph models can represent directly.

We therefore represent the controller state as a heterogeneous graph whose nodes are pending vehicles, committed vehicles, and the conflict zones, and whose seven typed edge relations encode which zones a vehicle will use, which vehicles share a lane, which pending vehicles conflict, and which committed vehicles block which pending ones. A relational graph convolutional network (R-GCN) [1] processes this graph, and an actor–critic head trained with PPO [2] selects either a feasible vehicle to admit or a deferral. Graph neural networks have been used for job-shop and dispatching problems [3–5] and for AIM [6], but, to our knowledge, there is no controlled comparison of a learned relational policy against standard dispatching rules and a receding-horizon exact reference under one shared feasibility protocol for the gap-free model [TODO: literature check for this novelty statement].

The results are mixed, and we report them as such. Averaged over eight arrival rates, the R-GCN lowers mean waiting time by 10.6% relative to iGreedy, 14.4% relative to Backpressure, and 27.5% relative to LIFO. The gain is concentrated at high load: the R-GCN is best at every rate from $\lambda = 0.65$ vehicles/s upward, but it is worse than iGreedy at the four lightest loads because it defers vehicles that could have been admitted immediately. Its main value in this paper is as a compact learned baseline, with about 5.1×10^5 parameters, against which larger relational architectures can be measured.

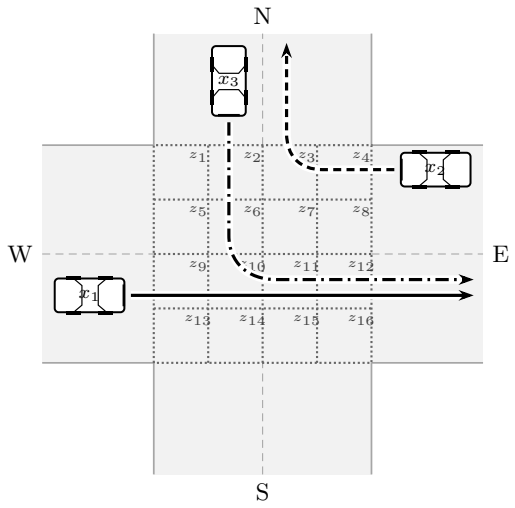


Figure 1: Traffic-oriented view of the 4×4 conflict-zone environment. Three approaching vehicles illustrate a straight movement (x_1, solid), a right turn (x_2, dashed), and a left turn (x_3, dash–dot). Dotted boundaries identify the 16 mutually exclusive zones while remaining visually subordinate to the roadway and vehicle trajectories.

Section 2 formalizes the gap-free admission problem; Section 3 situates the work within the adjacent literature; Section 4 states our contributions once both are in view.

2 Problem Formulation

2.1 Environment and Notation

The environment is a 4×4 intersection grid decomposed into a set of conflict zones $\mathcal{Z}$, $|\mathcal{Z}| = 16$ (Fig. 1). Let $\mathcal{V}$ denote the set of vehicles observed in an episode, indexed $i = 1, \ldots, N$. Each vehicle i has a fixed route $R_i = (z_i^1, z_i^2, \ldots, z_i^{n_i})$, an ordered sequence of zones it must traverse, and an arrival time a_i at the first zone. Traversing zone z_i^k takes a fixed free-flow processing time p_i^k determined by the vehicle's speed and the zone's geometry. Define the route offset

$$\delta_i^k = \sum_{j<k} p_i^j, \qquad \delta_i^1 = 0. \tag{1}$$

Gap-free admission. A vehicle is admitted by choosing a single departure time s_i. Its k-th zone-visit then occupies the half-open interval

$$I_i^k = [\, s_i + \delta_i^k,\ s_i + \delta_i^k + p_i^k \,), \tag{2}$$

so consecutive zone visits of one vehicle are back-to-back, with no holding inside the intersection. A feasible schedule is an assignment $\{s_i\}$ satisfying

$$s_i \geq a_i, \tag{3}$$

$$I_i^k \cap I_j^l = \emptyset \quad \text{whenever } z_i^k = z_j^l,\ i \neq j, \tag{4}$$

$$s_j \geq s_i + p_i^1 \quad \text{if } i \text{ precedes } j \text{ in the same lane.} \tag{5}$$

Constraint (3) releases a vehicle no earlier than its arrival; (4) is the disjunctive zone-exclusivity constraint of job-shop scheduling, applied to the shifted intervals (2); (5) makes same-lane first-in-first-out order a hard constraint and prevents a follower from entering its first zone before its leader has cleared it. The model is a job-shop-like instance in which vehicles are jobs and zones are machines, consistent with prior formulations of intersection coordination as job-shop scheduling [7]. Operations on disjoint zones overlap in time, so the environment is parallel across zones rather than serialized through one critical section. Because every vehicle has exactly one free variable, the decision of the controller is which vehicle to fix next, and at which earliest feasible start.

For a pending vehicle j, the *effective start* is the earliest time at which it may be admitted given its same-lane leaders,

$$s_j^{\text{eff}} = \max\Big(t,\ a_j,\ \max_{i \text{ leads } j} (s_i + p_i^1)\Big), \tag{6}$$

where t is the current time. The *admission check* asks whether the whole of (2), evaluated at $s_j = s_j^{\text{eff}}$, fits between the reservations already held; it returns a feasibility flag and, if the answer is negative, the next time at which the vehicle may become admissible.

2.2 Online Information Model

Unlike an offline instance, $\mathcal{V}$, $\{R_i\}$, and $\{a_i\}$ are not known in advance. A vehicle i is *detected* only at time $a_i - \Delta_{\text{det}}$, where $\Delta_{\text{det}} = 10\,\text{s}$. For a decision time t, the detected set is

$$\mathcal{D}(t) = \{\, i \in \mathcal{V} : a_i - \Delta_{\text{det}} \leq t \,\}. \tag{7}$$

Detected vehicles are PENDING until admitted. An admitted vehicle is TENTATIVE and becomes LOCKED, with an irrevocable s_i, once it lies within a commit window $\Delta_{\text{com}} = 2.5\,\text{s}$ of its departure:

$$\mathcal{L}(t) = \{\, i : s_i - t \leq \Delta_{\text{com}} \,\}. \tag{8}$$

Same-lane order is enforced at both transitions: a follower can be admitted only after all of its leaders have been admitted, and can be locked only after all of its leaders are locked.

Whenever a new vehicle is detected, every TENTATIVE admission is revoked and returned to the pending pool, which is then re-admitted by the active policy ("wipe-and-rerank"); locked admissions are never changed. At each decision point the policy chooses one pending vehicle from those that pass the admission check. The learned

policy has one further action, DEFER, which admits nobody and advances time by 0.5 s; to prevent starvation, an admission is forced after 2.0 s of consecutive deferrals. All other policies are driven through the identical loop and differ *only* in which feasible vehicle they select, so detection timing, revocation, and commit-time freezing are held fixed and the selection rule is the sole independent variable.

2.3 Objective

For each vehicle, waiting time is the admission delay

$$w_i = s_i - a_i = f_i - \Big(a_i + \sum_{k=1}^{n_i} p_i^k\Big), \tag{9}$$

where f_i is the finish time in the last zone, i.e., completion time in excess of the free-flow traversal time. The episode objective averages (9) over all observed vehicles:

$$W = \frac{1}{|\mathcal{V}|} \sum_{i \in \mathcal{V}} w_i. \tag{10}$$

Episodes last $T = 60$ s. A tentative admission at the end of the episode is finalized at its planned departure, and a vehicle that has not been admitted contributes $T - a_i$. All vehicles, including those still in flight, are therefore counted, and an unfinished backlog is penalized rather than hidden.

2.4 The Restricted-Information Reference Point

To provide a strong comparison under the information available to every online policy, we construct a receding-horizon reference point rather than an offline optimum computed with advance knowledge of all arrivals.

At each detection or retry event t_j, define the restricted-information subproblem $P(t_j)$: minimize (10), restricted to vehicles in $\mathcal{D}(t_j)$, subject to (3)–(5), with the start of every vehicle in $\mathcal{L}(t_j)$ fixed at its committed value. We solve $P(t_j)$ with CP-SAT, using one integer departure variable per vehicle, a no-overlap constraint per zone, and the same-lane precedence constraints. A vehicle is committed when its solved departure equals $\max(t_j, a_i)$, and vehicles are locked under the same commit-window rule used by every other policy. We denote the realized episode objective of the resulting sequence of sub-solves by W^*. If any sub-solve is not certified optimal within its time limit, the episode is discarded, so W^* is a mean over certified episodes only.

We are careful about what this construction establishes. Each solved $P(t_j)$ is exact within that subproblem; re-solving the visible subproblem at each event is a receding-horizon construction in which a decision is optimal given only the vehicles visible at t_j. This is directly analogous to a design choice in this problem class: Huang et al. [8] decompose a continuous vehicle stream into fixed-size batches, solve each batch to true CP-SAT optimality, and note that batch decomposition itself may lose the optimal solution of the full stream, so their optimality claim is scoped to *within* a batch. We adopt the same scoping discipline and report W^* as a rigorous *receding-horizon reference point*, not a proven episode-level optimum or lower bound. By construction, W^* never observes vehicle i before $a_i - \Delta_{\text{det}}$, and once a vehicle is locked it is as irrevocable for W^* as for any heuristic.

3 Related Work

This study is application-focused: it evaluates a learned relational policy within a multi-zone AIM environment against strong non-learned comparators, rather than proposing a new general scheduling theory. We organize the closest work by intersection-control mechanism and by its relationship to the evaluated policies. Broad reviews of cooperative, reservation-based, decentralized, and mixed-autonomy intersection management are available in [9–11].

Reservation and priority-based AIM: Dresner and Stone established the canonical space–time reservation framework using first-come-first-served admission [12, 13]. Later work introduced explicit priority mechanisms. Zhang et al. developed an event-triggered, state-driven reservation controller and the sPriorFIFO scheduler for vehicles with different urgency levels [14]. Dai et al. assigned priorities within collision areas while considering travel time and fairness [15], and Zhang et al. proposed a priority-based AIM scheme for connected automated vehicles [16]. The gap-free admission model used here is a reservation model in this tradition, in which a whole route is reserved or the request is delayed, and the policies studied differ only in the order in which pending requests are served.

Multi-zone scheduling and optimization: Conflict-point and job-shop models capture the fact that nonconflicting parts of an intersection can be used concurrently. Colombo and Del Vecchio studied collision avoidance over intersection conflict regions [17], while Ahn and Del Vecchio formulated multi-conflict-point coordination as a job-shop scheduling problem [7]. Related application-level controllers include rolling-horizon conflict-point reservation [18], MILP scheduling of desired access times [19], decentralized energy-optimal coordination [20], sliding-window lane and entry-time optimization [21], and lane-free batch processing [22]. Our reference point W^* follows the receding-horizon exact-solve pattern of this family, implemented with CP-SAT [23].

Learned AIM and the iGreedy comparator: Learning-based controllers have been applied to centralized vehicle coordination, mixed-traffic intersection management, and conflict-zone scheduling [6, 24, 25]. Huang et al. formulate intersection management as a constrained

job shop, train a learned dispatcher with reinforcement learning, and compare it with iGreedy, an earliest-achievable-start policy [8]; we adopt that comparator and naming. Recent benchmark work further emphasizes controlled comparisons among AIM policies [26]. These studies motivate comparing a learned policy against several inexpensive, application-specific rules before attributing an improvement to model or training complexity.

Graph and reinforcement learning for scheduling: Graph neural networks have become a standard way to encode scheduling instances whose structure varies from episode to episode. Zhang et al. learn priority dispatching rules for job-shop scheduling with a graph encoder and RL [3], Park et al. use a GNN to represent the disjunctive graph of a job shop [4], Mao et al. use a graph encoder for job scheduling in data-processing clusters [27], and Cappart et al. survey graph models for combinatorial optimization [5]. We use the relational graph convolution of Schlichtkrull et al. [1], which keeps a separate weight matrix per relation and extends the graph convolution of Kipf and Welling [28] to typed edges, together with PPO [2] and generalized advantage estimation [29].

Dispatching rules and benchmark scope: Due-date and operation-level priority rules are well established in job-shop scheduling [30–32]. Backpressure/MaxWeight provides the basis for queue-state-aware scheduling [33], and polling models provide a related queueing view of unsignalized intersection coordination [34].

Accordingly, the experiments evaluate the learned policy against three non-learned online policies under one detection, feasibility, and commitment protocol: iGreedy, LIFO, and a Backpressure-style policy.

4 Contributions

Relative to the problem formalized in Section 2 and the gap identified in Section 3, this paper makes the following contributions:

- We formulate gap-free admission as online scheduling with one departure variable per vehicle, hard same-lane order, and a commitment window, and define a heterogeneous graph with seven typed relations that exposes pending vehicles, committed vehicles, and conflict zones to a learned policy.
- We train an R-GCN actor–critic with PPO, a feasibility mask, and a separate DEFER head. The policy has 5.1×10^5 parameters and trains at 0.37 s per episode, measured as wall-clock time per training episode including simulation.
- We conduct a controlled, paired comparison of the R-GCN with iGreedy, LIFO, and Backpressure across eight demand levels, together with a receding-horizon CP-SAT reference. All policies use identical detection, feasibility, revocation, and commitment rules, and we report where the learned policy wins and where it loses.

The intended contribution is therefore a reproducible learned baseline and a comparative application study, not a claim of a dominant controller. The comparators precede this work [8, 30, 33].

5 Baseline Policies

All three non-learned policies operate on the set of pending vehicles that pass the admission check of Section 2. After a policy selects a vehicle, the common controller reserves its route and applies the same exclusivity, same-lane, and commitment-window logic. Thus the comparison changes the selection rule, not the safety or feasibility layer.

iGreedy: Among the feasible pending vehicles, iGreedy admits the one with the earliest effective start s_j^{eff} in (6), with ties broken by arrival time and vehicle identifier. We use the name and comparator role established in job-shop-formulated AIM work [8].

LIFO: LIFO admits the most recently arrived feasible vehicle.

Backpressure: Adapted from wireless-link scheduling [33]. Because nothing queues in the gap-free model (a vehicle is either admitted or waits outside with no reservation), there is no literal zone backlog. The adapted rule admits the feasible vehicle whose route passes through the most zones that are also wanted by other pending vehicles, i.e., it clears the most contested demand first, with ties broken by earlier arrival. It is throughput-oriented by design rather than delay-oriented.

6 R-GCN Admission Policy

6.1 Graph Representation

At every decision point the controller state is encoded as a heterogeneous graph with three node types. *Pending* nodes are detected, not-yet-admitted vehicles and are the action candidates. *Committed* nodes are tentative or locked vehicles whose reservation has not yet ended. *Zone* nodes are the 16 conflict zones. All time features are measured relative to the current time t and divided by a fixed scale of 10 s so that they remain $O(1)$ across arrival rates.

A pending node carries 30 features: ten scalars (a feasibility flag, whether all leaders are admitted, the offset of s_j^{eff} from t, the arrival offset, the wait so far, the retry-time offset, the number of pending leaders, route length, total free-flow time, and the number of committed blockers), a 16-way manoeuvre one-hot, and four global features (elapsed episode fraction and the numbers of pending, tentative, and committed vehicles). A committed node carries 21 features (locked flag, departure and finish offsets, arrival offset, route length, manoeuvre one-hot). A zone node carries 21 features (time until free,

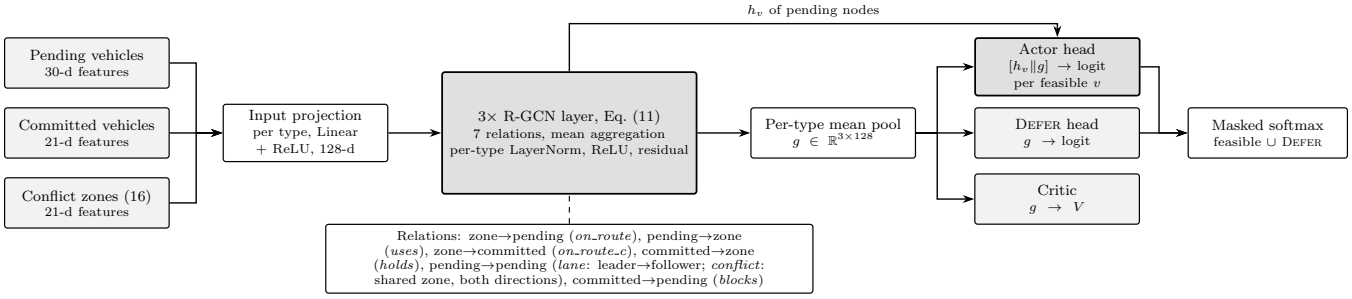


Figure 2: R-GCN admission policy. Node features of the three types are projected to a common width and refined by three R-GCN layers over the seven typed relations. Per-type mean pooling gives a graph summary g for the DEFER head and the critic; the actor scores each pending vehicle from its own embedding and g. Infeasible vehicles are masked before the softmax.

occupancy flag, load over the next 10 s, number of future reservations, number of pending vehicles that want the zone, and a zone one-hot).

Seven relations connect the nodes: *on_route* (zone→pending), *uses* (pending→zone), *on_route_c* (zone→committed), *holds* (committed→zone), *lane* (pending leader→pending follower), *conflict* (pending↔pending vehicles whose routes share a zone), and *blocks* (committed blocker→pending vehicle). The graph is rebuilt at every decision, so its size follows the number of vehicles currently visible.

6.2 Encoder and Heads

Each node type is first projected to a common width of 128 by a type-specific linear layer. Message passing uses three R-GCN layers [1], with a separate weight matrix W_r per relation, mean aggregation within a relation, and a self-loop matrix W_0,

$$h_i' = W_0\, h_i + \sum_{r=1}^{7} \frac{1}{|\mathcal{N}_r(i)|} \sum_{j \in \mathcal{N}_r(i)} W_r\, h_j. \qquad (11)$$

The three node types are flattened into one index space so that a single convolution handles all relations. After each layer a ReLU, a residual connection, and a type-specific layer normalization [35] are applied. Per-type mean pooling of the final embeddings gives a 3×128 graph summary g.

The actor is a two-layer perceptron (hidden width 64) applied to the concatenation $[h_v \| g]$ of a pending vehicle's embedding and the graph summary, and produces one logit per pending vehicle. Infeasible vehicles are masked. A separate DEFER head (hidden width 64) maps g to a single logit, whose output bias is initialized to -2 so that early training behaves like an order-only rule rather than deferring at random. A critic perceptron (hidden width 128) maps g to a state-value estimate. The policy has 512,515 trainable parameters.

6.3 Actions and Reward

At a decision point the policy selects either one feasible pending vehicle, which is admitted at its effective start, or DEFER, which advances time by 0.5 s. After 2.0 s of consecutive deferrals an admission is forced. Because the mask removes infeasible vehicles, the policy cannot propose a reservation that violates (3)–(5).

The waiting time of a vehicle is final once it is locked or the episode ends. At that point the policy receives the reward $-\kappa\, w_i/N$ with $\kappa = 10$, so the undiscounted episode return equals $-\kappa W$. To reduce variance, the return is measured against iGreedy on the same scenario, i.e., it is $\kappa\,(W_{\text{iGreedy}} - W_\pi)$, which is positive when the policy improves on iGreedy.

6.4 PPO Training

The policy is trained with PPO [2] using generalized advantage estimation [29]. Each update collects 32 episodes whose arrival rate is drawn uniformly from $[0.1, 1.2]$ vehicles/s and runs four epochs of minibatch updates. The entropy coefficient is annealed from 0.01 to 0.001. Table 1 lists all hyperparameters. The model is implemented with PyTorch Geometric [36].

7 Experimental Setup

7.1 Intersection and Traffic Model

Geometry and routes. The simulation uses the 4×4 conflict-zone intersection shown in Fig. 1. The 16 zones are indexed on the normalized lattice $(x, y) \in \{0, 1, 2, 3\}^2$. Four approaches (north, south, east, and west), each with two incoming lanes, define eight same-lane queues. For the north, south, east, and west approaches, respectively, the entry-zone pairs are $\{z_1, z_2\}$, $\{z_{15}, z_{16}\}$, $\{z_4, z_8\}$, and $\{z_9, z_{13}\}$; the corresponding exit-zone pairs are $\{z_3, z_4\}$, $\{z_{13}, z_{14}\}$, $\{z_1, z_5\}$, and $\{z_{12}, z_{16}\}$.

The shared crossing area is $\mathcal{Z}_{\text{core}} = \{z_6, z_7, z_{10}, z_{11}\}$. Its zones have effective traversal length 7.0 normalized distance units, whereas each of the 12 boundary entry/exit

Table 1: R-GCN policy and PPO hyperparameters. †[TODO: confirm episode count and checkpoint]

Hidden width / R-GCN layers	128 / 3
Relations / aggregation	7 / mean
Actor, DEFER, critic hidden width	64, 64, 128
DEFER bias init.	-2.0
Parameters	512,515
Learning rate	3×10^{-4}
Discount γ / GAE λ	1.0 / 0.95
Clip ϵ	0.2
Epochs / minibatch	4 / 256
Value coefficient	0.5
Entropy coefficient	$0.01 \to 0.001$
Gradient clip	0.5
Episodes per update	32
Training arrival rate	$\mathcal{U}(0.1, 1.2)$
Reward scale κ	10
Configured training episodes	15,000
Episodes completed	$\approx$4,350†

zones has length 5.0:

$$\ell(z) = \begin{cases} 7.0, & z \in \mathcal{Z}_{\text{core}}, \\ 5.0, & z \in \mathcal{Z} \setminus \mathcal{Z}_{\text{core}}. \end{cases} \tag{12}$$

The manoeuvre set $\mathcal{R}$ contains 16 routes: eight straight movements containing four zones each, four right turns containing two zones each, and four left turns containing five zones each. Routes are sampled uniformly and independently,

$$\Pr(R_i = r) = \frac{1}{16}, \qquad r \in \mathcal{R}. \tag{13}$$

No empirical route weighting is used.

Vehicle and operation times. Each vehicle receives one independent speed draw $v_i \sim \mathcal{U}(8.0, 14.0)$ in normalized distance units per second, held fixed across its route. For the visit to zone z_i^k, the processing time is

$$p_i^k = \frac{\ell(z_i^k)}{v_i} + \epsilon_i^k, \qquad \epsilon_i^k \sim \mathcal{U}(0, 0.3\,\text{s}), \tag{14}$$

with perturbations sampled independently across operations. The route, speed, and operation-time perturbations are synthetic draws rather than calibrated quantities.

Arrivals and headway. Vehicle arrivals follow one aggregate Poisson process for the complete intersection. The tested aggregate rates are

$$\lambda \in \{0.10, 0.20, 0.35, 0.50, \\ 0.65, 0.80, 1.00, 1.20\} \text{ vehicles/s}. \tag{15}$$

Each episode lasts $T = 60\,\text{s}$, so $\mathbb{E}[N] = \lambda T$ ranges from 6 to 72 vehicles per episode. Realized vehicle counts are reported in Table 2. The range covers light through increasingly congested demand and was informed by aggregate arrival-rate observations from the SinD trajectory dataset [37]; the experiments do not replay SinD trajectories, and interarrival times remain synthetic and memoryless. The training distribution $\mathcal{U}(0.1, 1.2)$ covers the same range, so the evaluation measures interpolation within the training range, not extrapolation.

The generator used for the rate sweep does not impose a minimum same-lane headway. Vehicles in the same lane may therefore arrive arbitrarily close together, which avoids granting the scheduler guaranteed temporal spacing and provides a conservative stress test.

7.2 Common Online Controller

All policies are executed by the event-driven controller of Section 2. A vehicle becomes visible $\Delta_{\text{det}} = 10\,\text{s}$ before its arrival, and a tentative admission becomes irrevocable inside the $\Delta_{\text{com}} = 2.5\,\text{s}$ commitment window. Each new detection revokes all tentative admissions and readmits the pending pool. The only experimental factor among iGreedy, LIFO, Backpressure, and the R-GCN is the choice among feasible vehicles (and, for the R-GCN, the option to defer). Each traffic instance is frozen before evaluation and replayed unchanged for every policy, so arrival times, routes, processing times, and tie-breaking identifiers are paired across policies.

7.3 Scaling Experiment and Reporting

For every arrival rate, we generate three independent replicates (seeds 1, 2, 3) of 15 independently sampled 60 s episodes, giving 45 paired episodes per rate and policy. The R-GCN acts deterministically (argmax over the masked logits). The primary response is the mean per-vehicle waiting time W in (10), averaged over the 45 episodes. For two policies A and B, the reported reduction of A relative to B is

$$100\,\frac{\bar{W}_B - \bar{W}_A}{\bar{W}_B}\ \%, \tag{16}$$

with both means evaluated on the same instances. Win/tie/loss counts compare the R-GCN with iGreedy episode by episode, with a tie when $|\Delta W| \le 10^{-6}\,\text{s}$. [TODO: state whether evaluation seeds 1–3 overlap with the seeds used for checkpoint selection during training] Evaluation was run with one trained model; variability due to training seeds is not measured.

7.4 Implementation and Runtime Environment

The simulator and policies are implemented in Python and PyTorch with PyTorch Geometric [36]; the receding-horizon reference uses OR-Tools CP-SAT [23]. CP-SAT accepts integer-valued time variables, so continuous times

Table 2: Mean per-vehicle waiting time (s) versus arrival rate (45 paired episodes per rate and policy). Avg. n veh. is the mean number of vehicles per episode. Bold marks the lowest value in each row. The bottom row is the unweighted mean over the eight rates.

Rate (veh/s)	Avg. n veh	iGreedy	B.P.	LIFO	R-GCN (ours)
0.10	6.2	**0.030**	0.039	0.075	0.053
0.20	11.2	**0.074**	0.085	0.142	0.150
0.35	21.8	**0.126**	0.145	0.208	0.163
0.50	29.8	**0.273**	0.291	0.377	0.287
0.65	38.6	0.374	0.413	0.525	**0.344**
0.80	46.6	0.512	0.555	0.620	**0.438**
1.00	58.9	0.730	0.782	0.863	**0.587**
1.20	71.1	0.979	0.924	1.008	**0.746**
Mean	–	0.387	0.404	0.477	**0.346**

are mapped to integers with scale factor $S = 100{,}000$ (a resolution of 10^{-5} s). Each reference solve uses one search worker and a 5 s limit. [TODO: state Python, PyTorch, PyTorch Geometric and OR-Tools versions and the hardware used]

7.5 Receding-Horizon Reference Experiment

The W^* experiment re-solves the visible subproblem at every detection or retry event, as in Section 2.4. It is run on the same scenario stream as the other policies. Episodes in which any sub-solve was not certified optimal within the limit are dropped, so W^* is the mean over certified episodes only and its sample differs from the 45-episode samples. [TODO: record and report the number of certified episodes per rate]

8 Results

8.1 Scaling Study: Waiting Time versus Arrival Rate

Table 2 reports the mean episode-level waiting time of the four online policies and Fig. 3 shows the same results graphically. Mean waiting time increases with demand for every policy. The R-GCN rises from 0.053 s at $\lambda = 0.10$ to 0.746 s at $\lambda = 1.20$; over the same range iGreedy rises from 0.030 to 0.979 s, Backpressure from 0.039 to 0.924 s, and LIFO from 0.075 to 1.008 s. Averaged over the eight rates, the R-GCN is lowest (0.346 s), followed by iGreedy (0.387 s), Backpressure (0.404 s), and LIFO (0.477 s).

The ranking depends on load. At the four lightest rates ($\lambda \le 0.50$) iGreedy has the lowest mean, and the R-GCN is worse than iGreedy by 0.023, 0.076, 0.038, and 0.015 s. From $\lambda = 0.65$ onward the R-GCN is the best policy, and its advantage grows with load, from 0.030 s per vehicle at $\lambda = 0.65$ to 0.233 s at $\lambda = 1.20$.

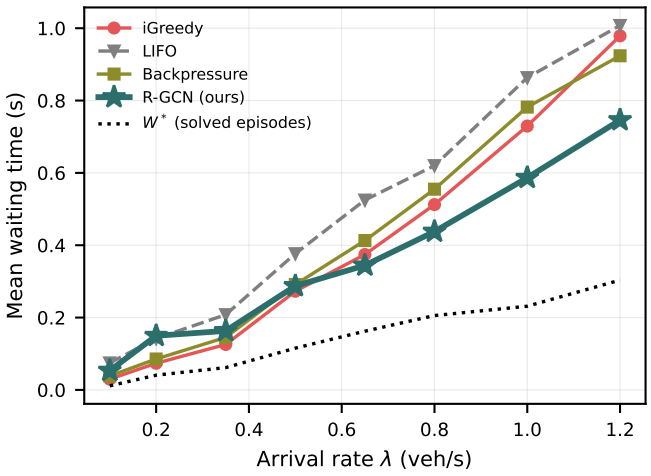


Figure 3: Mean waiting time versus arrival rate. The R-GCN (teal) is above iGreedy (red) at low load, crosses it between $\lambda = 0.50$ and 0.65, and grows more slowly at high load. The dotted line is the receding-horizon reference W^*, computed on the subset of episodes in which every CP-SAT subproblem was certified optimal.

8.2 Reduction Relative to the Baselines

Table 3 gives the relative reductions. Averaged over rates, the R-GCN lowers waiting time by 10.6% relative to iGreedy, 14.4% relative to Backpressure, and 27.5% relative to LIFO. The reduction relative to iGreedy is -103.2% to -5.3% for $\lambda \le 0.50$ and $+8.0\%$ to $+23.8\%$ for $\lambda \ge 0.65$. Relative to Backpressure, the R-GCN is worse at $\lambda \le 0.35$ and better from $\lambda = 0.50$ upward ($+1.4\%$ to $+25.0\%$). Relative to LIFO, it is better at every rate except $\lambda = 0.20$ (-5.4%). Because each policy receives identical arrivals, routes, and operation times and uses the same feasibility controller, these differences are attributable within the experiment to the selection rule.

The win/tie/loss counts show the same pattern at the episode level. At $\lambda = 0.10$ the R-GCN ties iGreedy in 43 of 45 episodes, since few vehicles ever contend. At $\lambda = 0.20$ it wins 2, ties 25, and loses 18 episodes, and at $\lambda = 1.20$ it wins 38 and loses 7. The percentage reductions at light load are large because the baseline waiting times are tiny (0.030 s at $\lambda = 0.10$), and should be read together with the absolute differences above.

8.3 Deferral and the Receding-Horizon Reference

Table 4 reports the receding-horizon reference W^* and the fraction of R-GCN decisions that chose DEFER. The R-GCN defers in 37% of its decisions at $\lambda = 0.10$ and

Table 3: Waiting-time reduction of the R-GCN relative to each baseline, $100(\bar{W}_B - \bar{W}_{\text{R-GCN}})/\bar{W}_B$ (%; negative values mean the R-GCN is worse), and the R-GCN's win/tie/loss count against iGreedy over the 45 paired episodes (ties: $|\Delta W| \leq 10^{-6}$ s).

Rate	iGreedy	B.P.	LIFO	W/T/L vs. iGreedy
0.10	−78.7	−38.0	+28.5	0/43/2
0.20	−103.2	−75.2	−5.4	2/25/18
0.35	−29.9	−12.4	+21.6	10/12/23
0.50	−5.3	+1.4	+23.7	19/2/24
0.65	+8.0	+16.6	+34.4	29/0/16
0.80	+14.6	+21.1	+29.4	28/0/17
1.00	+19.6	+25.0	+32.1	33/0/12
1.20	+23.8	+19.2	+26.0	38/0/7
Mean	+10.6	+14.4	+27.5	–

Table 4: Receding-horizon reference W^* and R-GCN deferral rate. W^* is the mean over the episodes in which every CP-SAT subproblem was certified optimal within 5 s, so its sample differs from the 45-episode means in Table 2 and is not directly comparable to them. "Defer" is the fraction of R-GCN decisions that chose DEFER.

Rate (veh/s)	W^* (s)	Defer
0.10	0.011	0.37
0.20	0.041	0.48
0.35	0.062	0.50
0.50	0.115	0.51
0.65	0.162	0.62
0.80	0.206	0.64
1.00	0.231	0.69
1.20	0.304	0.71

in 71% at $\lambda = 1.20$, and the deferral fraction increases monotonically with load. Deferral is an action the baselines do not have, so the share of decisions is not a direct measure of idling. We have not measured whether the vehicles deferred at low load could have been admitted immediately at no cost to others, and so report the over-deferral at low load as an observed performance deficit rather than a diagnosed cause.

W^* rises from 0.011 s at $\lambda = 0.10$ to 0.304 s at $\lambda = 1.20$ and is below the mean of every online policy at every rate. Because the W^* sample is restricted to certified episodes and the other policies are averaged over all 45 episodes, the two are not paired, so the difference should be read as an indication of remaining headroom rather than an optimality gap. As in the earlier receding-horizon constructions discussed in Section 2.4, W^* is not an episode-level optimum, since it too operates without knowledge of future arrivals.

9 Discussion

9.1 Where the Learned Policy Helps

The R-GCN gains most where contention is highest. At $\lambda \geq 0.65$ the pending pool is large, conflicts between pending vehicles are frequent, and the order of admissions has a measurable effect on downstream waiting. In that regime the policy sees, through the *conflict*, *lane*, and *blocks* relations, information that the three heuristics do not use jointly: iGreedy considers only each vehicle's own effective start, LIFO considers only recency, and Backpressure considers only route overlap with other pending vehicles. The R-GCN's advantage over all three at high load is consistent with learned use of this relational structure, but our experiments do not isolate which relations matter, and we make no such claim without an ablation.

9.2 Where the Learned Policy Hurts

At light load there is almost no contention, so the best action is almost always to admit the vehicle with the earliest effective start, which is exactly what iGreedy does. The R-GCN is slightly worse at these rates, and its deferral fraction is already 37% at $\lambda = 0.10$. A plausible explanation is that the policy was trained on rates drawn uniformly from $[0.1, 1.2]$ and that most gradient signal, and most of the achievable improvement over iGreedy, comes from congested episodes. Because the reward is measured against iGreedy, a policy that matches iGreedy at light load receives no signal there, while a few mistakes there are not offset. This is a hypothesis, not a tested finding. Two simple remedies are available and untested: fall back to iGreedy when few vehicles are pending, or condition the DEFER head on load.

9.3 Engineering Significance

The practical result is bounded. Within this benchmark, a 0.5-million-parameter relational policy trained for a fraction of its configured budget beats three standard dispatching rules on average and by a clear margin at high load, but it does not beat the simplest rule, iGreedy, at low load. Since AIM benefits are most valuable under congestion, the high-load behaviour is the more relevant one, yet the low-load deficit is real and any deployment would need the fallback mentioned above. The result also shows the value of reporting learned controllers against several non-learned rules: relative to LIFO alone the R-GCN looks excellent at every load, whereas relative to iGreedy it loses at four of eight rates.

9.4 Deployment Implications

The learned policy does not change the feasibility layer, so any deployment can keep a rule-based controller as a fallback and use the policy only to choose among already-feasible admissions. The policy requires the graph to be

rebuilt and evaluated at each decision, which costs more than a rule such as iGreedy; we have not measured inference latency in a controlled setting. [TODO: add a table of per-decision inference latency for R-GCN and each baseline] The policy also depends on reliable detection, declared routes, nominal processing times, and a consistent time reference. Route or speed estimates may change after detection, packets may be delayed or lost, and clocks may not be synchronized. The detection horizon Δ_{det} and commitment window Δ_{com} are held fixed here, and the sensitivity of the learned policy to either is unknown.

9.5 Safety, Feasibility, and Robustness

Safety and deadlock freedom within the modeled abstraction are properties of the shared feasibility layer, not benefits of the learned policy. The mask guarantees that the policy only proposes reservations that satisfy zone exclusivity and same-lane order, and the forced admission after 2.0 s of deferral bounds the delay that the DEFER action can add. The zone model represents occupancy through nonoverlapping intervals with no clearance buffer, and does not model acceleration limits, braking uncertainty, tracking error, vehicle dimensions, or continuous trajectories. "Safe" here means feasible with respect to the discrete zone-occupancy model, not certified collision avoidance for physical vehicles. A physical implementation would require reservation margins, vehicle-following constraints, emergency fallback behaviour, and procedures for vehicles that arrive late, stop, or change route.

9.6 Limitations and External Validity

The first limitation is training. The reported model was produced by a run that was interrupted at roughly 4,350 of the 15,000 configured episodes, and the reported numbers come from the checkpoint that was available at that point. [TODO: retrain to the full budget, state whether the best-evaluation or last checkpoint is used, and re-run all experiments] The reported results may therefore differ from those of a fully trained policy. Training variability across seeds is not measured, and no ablation of the seven relations, the DEFER action, or the iGreedy baseline in the reward is performed.

The second concerns traffic demand. Although the rate range is informed by observations from the SinD dataset, the experiments do not replay measured trajectories. Arrivals follow a single aggregate Poisson process, routes are drawn uniformly from 16 manoeuvres, and speeds and per-zone perturbations are independently sampled. The streams therefore lack platoons, time-varying demand, correlated approach flows, and observed turning proportions. Real intersections often exhibit directional imbalance, and a policy that performs well under balanced demand may behave differently when one approach is persistently oversubscribed.

The third concerns scope. The experiments use one isolated 4×4 geometry, a fixed route pool, fully connected vehicles, and a 60 s episode. They cover no mixed traffic, no pedestrians, no heterogeneous vehicle dynamics, and no long-run queue stability. The comparison includes three baselines only; a stronger non-learned controller, including one that repairs tentative admissions, could narrow or reverse the high-load advantage, and we have not tested this. Finally, W^* is a same-information receding-horizon reference computed on a certified subset, not a clairvoyant optimum or a paired lower bound.

Taken together, the experiments support a deliberately bounded conclusion: within the reported isolated, synthetic, fully connected simulation, a compact R-GCN admission policy reduces mean waiting time relative to iGreedy, Backpressure, and LIFO on average and at high load, while remaining worse than iGreedy under light traffic.

10 Conclusion

This paper introduced an R-GCN policy for gap-free admission in autonomous intersection management. The controller state is a heterogeneous graph of pending vehicles, committed vehicles, and conflict zones with seven typed relations; an actor–critic head trained with PPO either admits a feasible vehicle or defers, under an unchanged feasibility and commitment layer. Over eight arrival rates and 45 paired episodes per rate, the policy lowered the rate-averaged waiting time by 10.6% relative to iGreedy, by 14.4% relative to Backpressure, and by 27.5% relative to LIFO, and was the best policy at every rate from 0.65 vehicles/s upward, reaching a 23.8% reduction relative to iGreedy at 1.20 vehicles/s. At loads up to 0.50 vehicles/s it was worse than iGreedy.

The contribution is a reproducible, compact learned baseline and an honest comparison, not a claim of a dominant controller. The conclusions remain bounded by an incompletely trained model, an isolated synthetic intersection, and a small set of baselines. Future work includes completing training, ablating the relations and the deferral action, extending the comparison to stronger non-learned and larger learned controllers, and evaluating on measured traffic.

Acknowledgments

[To be added.]

References

[1] M. Schlichtkrull, T. N. Kipf, P. Bloem, R. van den Berg, I. Titov, and M. Welling, "Modeling relational data with graph convolutional networks," in *The Se-*

mantic Web (ESWC 2018), ser. Lecture Notes in Computer Science, vol. 10843, 2018, pp. 593–607.

[2] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," *arXiv preprint arXiv:1707.06347*, 2017.

[3] C. Zhang, W. Song, Z. Cao, J. Zhang, P. S. Tan, and C. Xu, "Learning to dispatch for job shop scheduling via deep reinforcement learning," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, 2020.

[4] J. Park, J. Chun, S. H. Kim, Y. Kim, and J. Park, "Learning to schedule job-shop problems: Representation and policy learning using graph neural network and reinforcement learning," *International Journal of Production Research*, vol. 59, no. 11, pp. 3360–3377, 2021.

[5] Q. Cappart, D. Chételat, E. B. Khalil, A. Lodi, C. Morris, and P. Veličković, "Combinatorial optimization and reasoning with graph neural networks," *Journal of Machine Learning Research*, vol. 24, no. 130, pp. 1–61, 2023.

[6] M. Klimke, B. Völz, and M. Buchholz, "Automatic intersection management in mixed traffic using reinforcement learning and graph neural networks," *arXiv preprint arXiv:2301.12717*, 2023.

[7] H. Ahn and D. Del Vecchio, "Semi-autonomous intersection collision avoidance through job-shop scheduling," in *Proc. 19th Int. Conf. on Hybrid Systems: Computation and Control (HSCC)*, 2016, pp. 185–194.

[8] S.-C. Huang, K.-E. Lin, C.-Y. Kuo, L.-H. Lin, M. O. Sayin, and C.-W. Lin, "Reinforcement-learning-based job-shop scheduling for intelligent intersection management," in *Proc. 2023 Design, Automation & Test in Europe Conf. (DATE)*, 2023.

[9] L. Chen and C. Englund, "Cooperative intersection management: A survey," *IEEE Transactions on Intelligent Transportation Systems*, vol. 17, no. 2, pp. 570–586, 2016.

[10] J. Rios-Torres and A. A. Malikopoulos, "A survey on the coordination of connected and automated vehicles at intersections and merging at highway on-ramps," *IEEE Transactions on Intelligent Transportation Systems*, vol. 18, no. 5, pp. 1066–1077, 2017.

[11] Z. Zhong, M. Nejad, and E. E. Lee, "Autonomous and semi-autonomous intersection management: A survey," *IEEE Intelligent Transportation Systems Magazine*, vol. 13, no. 2, pp. 53–70, 2021.

[12] K. Dresner and P. Stone, "Multiagent traffic management: A reservation-based intersection control mechanism," in *Proc. 3rd Int. Joint Conf. on Autonomous Agents and Multiagent Systems (AAMAS)*, 2004, pp. 530–537.

[13] K. Dresner and P. Stone, "A multiagent approach to autonomous intersection management," *Journal of Artificial Intelligence Research*, vol. 31, pp. 591–656, 2008.

[14] K. Zhang, D. Zhang, A. de La Fortelle, X. Wu, and J. Grégoire, "State-driven priority scheduling mechanisms for driverless vehicles approaching intersections," *IEEE Transactions on Intelligent Transportation Systems*, vol. 16, no. 5, pp. 2487–2500, 2015.

[15] P. Dai, K. Liu, Q. Zhuge, E. H.-M. Sha, V. C. S. Lee, and S. H. Son, "Quality-of-experience-oriented autonomous intersection control in vehicular networks," *IEEE Transactions on Intelligent Transportation Systems*, vol. 17, no. 7, pp. 1956–1967, 2016.

[16] H. Zhang, R. Zhang, C. Chen, D. Duan, X. Cheng, and L. Yang, "A priority-based autonomous intersection management scheme for connected automated vehicles," *Vehicles*, vol. 3, no. 3, pp. 533–544, 2021.

[17] A. Colombo and D. Del Vecchio, "Efficient algorithms for collision avoidance at intersections," in *Proc. 15th ACM Int. Conf. Hybrid Systems: Computation and Control (HSCC)*, 2012, pp. 145–154.

[18] M. W. Levin and D. Rey, "Conflict-point formulation of intersection control for autonomous vehicles," *Transportation Research Part C: Emerging Technologies*, vol. 85, pp. 528–547, 2017.

[19] S. A. Fayazi and A. Vahidi, "Mixed-integer linear programming for optimal scheduling of autonomous vehicle intersection crossing," *IEEE Transactions on Intelligent Vehicles*, vol. 3, no. 3, pp. 265–275, 2018.

[20] A. A. Malikopoulos, C. G. Cassandras, and Y. J. Zhang, "A decentralized energy-optimal control framework for connected automated vehicles at signal-free intersections," *Automatica*, vol. 93, pp. 244–256, 2018.

[21] W. Wu, Y. Liu, W. Liu, F. Zhang, V. Dixit, and S. T. Waller, "Autonomous intersection management for connected and automated vehicles: A lane-based method," *IEEE Transactions on Intelligent Transportation Systems*, vol. 23, no. 9, pp. 15091–15106, 2022.

[22] B. Li, Y. Zhang, T. Acarman, Y. Ouyang, C. Yaman, and Y. Wang, "Lane-free autonomous intersection

management: A batch-processing framework integrating reservation-based and planning-based methods," in *Proc. IEEE Int. Conf. Robotics and Automation (ICRA)*, 2021.

[23] L. Perron and V. Furnon, "OR-Tools," Google. [Online]. Available: https://developers.google.com/optimization

[24] Y. Guan, Y. Ren, S. E. Li, Q. Sun, L. Luo, and K. Li, "Centralized cooperation for connected and automated vehicles at intersections by proximal policy optimization," *IEEE Transactions on Vehicular Technology*, 2020.

[25] Y. Shi, W. Wang, X. Tao, I. Dusparic, and V. Cahill, "Applying neural Monte Carlo tree search to unsignalized multi-intersection scheduling for autonomous vehicles," *arXiv preprint arXiv:2410.18786*, 2024.

[26] L. Guo, L. Liu, J. Tang, B. Liu, and S. Cao, "OpenCDA-MARL: A unified benchmarking framework for cooperative autonomous intersection management with multi-agent reinforcement learning," *IEEE Robotics and Automation Letters*, vol. 11, no. 4, pp. 4291–4298, 2026.

[27] H. Mao, M. Schwarzkopf, S. B. Venkatakrishnan, Z. Meng, and M. Alizadeh, "Learning scheduling algorithms for data processing clusters," in *Proc. ACM SIGCOMM*, 2019, pp. 270–288.

[28] T. N. Kipf and M. Welling, "Semi-supervised classification with graph convolutional networks," in *Proc. Int. Conf. Learning Representations (ICLR)*, 2017.

[29] J. Schulman, P. Moritz, S. Levine, M. Jordan, and P. Abbeel, "High-dimensional continuous control using generalized advantage estimation," in *Proc. Int. Conf. Learning Representations (ICLR)*, 2016.

[30] S. S. Panwalkar and W. Iskander, "A survey of scheduling rules," *Operations Research*, vol. 25, no. 1, pp. 45–61, 1977.

[31] J. H. Blackstone, D. T. Phillips, and G. L. Hogg, "A state-of-the-art survey of dispatching rules for manufacturing job shop operations," *International Journal of Production Research*, vol. 20, no. 1, pp. 27–45, 1982.

[32] R. Haupt, "A survey of priority rule-based scheduling," *OR Spektrum*, vol. 11, pp. 3–16, 1989.

[33] L. Tassiulas and A. Ephremides, "Stability properties of constrained queueing systems and scheduling policies for maximum throughput in multihop radio networks," *IEEE Transactions on Automatic Control*, vol. 37, no. 12, pp. 1936–1948, 1992.

[34] D. Miculescu and S. Karaman, "Polling-systems-based autonomous vehicle coordination in traffic intersections with no traffic signals," *IEEE Transactions on Automatic Control*, vol. 65, no. 3, pp. 1527–1543, 2020.

[35] J. L. Ba, J. R. Kiros, and G. E. Hinton, "Layer normalization," *arXiv preprint arXiv:1607.06450*, 2016.

[36] M. Fey and J. E. Lenssen, "Fast graph representation learning with PyTorch Geometric," in *ICLR Workshop on Representation Learning on Graphs and Manifolds*, 2019.

[37] Y. Xu, W. Shao, J. Li, K. Yang, W. Wang, H. Huang, C. Lv, and H. Wang, "SinD: A drone dataset at signalized intersection in China," in *Proc. 2022 IEEE 25th Int. Conf. on Intelligent Transportation Systems (ITSC)*, 2022, pp. 2471–2478.